Reviewer Pack — Polymatroid Rank Gap in Monotone DNF Depth

Entry e3302ada2f9d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 20:16:28 UTC

Polymatroid Rank Gap in Monotone DNF Depth

Entry ID: `e3302ada2f9d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 20:16:28 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone Circuit Depth for k-CLIQUE

Statement:

> For any k-CLIQUE indicator function f on n vertices, the polymatroid rank function ρ_f of its hypergraph representation satisfies ρ_f ≥ Ω(n/k) while remaining ≤ O(log n) for all DNFs with size poly(n).

Rationale (proposer's reasoning):

> Polymatroid rank captures submodular structure of hypergraphs; its gap between k-CLIQUE and general DNF could expose inherent depth complexity. The conjecture links combinatorial optimization (polymatroids) to monotone circuit depth, bypassing traditional algebraic barriers.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e1dbe98c6d03f637`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        edges = []
        for i in range(k):
            for j in range(i + 1, k):
                edges.append((i, j))
        for i in range(k, n):
            for edge in edges:
                if edge[0] < k and edge[1] < k:
                    continue
                if random.choice([True, False]):
                    edges.append(edge)
        return edges

    def hypergraph_rank(edges, n):
        rank = [0] * n
        for u, v in edges:
            if rank[u] == 0 or rank[v] == 0:
                rank[u] += 1
                rank[v] += 1
        return max(rank)

    def is_dnf_formula(formula):
        return all(isinstance(clause, list) and all(isinstance(lit, int) for lit in clause) for cl

    n = random.randint(5, 40)
    k = random.randint(2, min(n // 2, 10))
    edges = generate_k_clique(n, k)
    rank_f = hypergraph_rank(edges, n)

    if rank_f < n / (4 * k):
        return {
            "metric_name": "polymatroid_rank",
            "metric_value": rank_f,
            "instances_tested": 1,
            "conjecture_holds": False,
```

```
            "counterexample": f"n={n}, k={k}, rank_f={rank_f} < n/(4*k)"
        }

    formula = []
    for _ in range(random.randint(5, 20)):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, 3
        formula.append(clause)

    if is_dnf_formula(formula):
        rank_f = hypergraph_rank([(abs(lit), lit // abs(lit)) for clause in formula for lit in cla
        if rank_f > 3 * math.log(n):
            return {
                "metric_name": "polymatroid_rank",
                "metric_value": rank_f,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, DNF size={len(formula)}, rank_f={rank_f} > 3*log(n)"
            }

    return {
        "metric_name": "polymatroid_rank",
        "metric_value": rank_f,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbf79bba.py", line 89, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbf79bba.py", line 65, in run_trial
   rank_f = hypergraph_rank([(abs(lit), lit // abs(lit)) for clause in formula for lit in clause], n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbf79bba.py", line 37, in hypergraph_ra
    if rank[u] == 0 or rank[v] == 0:
       ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with IndexError before collecting data; no results available to assess conjecture | next: Fix hypergraph_rank implementation to handle empty lists/indices properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	34001
2	preregistration	ollama_remote	qwen3:8b	0	0	19766
3	novelty	ollama_remote	qwen3:8b	0	0	16358
4	novelty	ollama_remote	qwen3:8b	0	0	12938
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17308
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10115
7	judge	ollama_remote	qwen3:8b	0	0	10828

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121314 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e3302ada2f9d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/e3302ada2f9d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e3302ada2f9d.tar.gz` (if generated)